

Class 6: R Functions

Dea Sinaga (PID: A17725676)

Table of contents

Background	1
Q1. Write your first R function: add()	1
Q2. Write a generate_dna() function	3
Q3. Write a generate_protein() function	5
Q4. Generate random protein sequences of length 6 to 13	6
Q5. BLASTp search against nr — are your peptides “unique in nature”?	7
Q6. Connecting your findings to immunology (MHC class II and T-cell activation)	7

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of Rn code that do the work of the function).

Q1. Write your first R function: add()

Here we will create a function to add some numbers. Let's call it 'add()'.

Input arguments can be either “**required**” or “**optional**”. The latter have fall-back default values that will be used if the user does not satisfy them.

Q1a. Your first version of the function should add two inputs together. For example, add (4, 7) should return 11.

```
add <- function(x, y=0) {  
  x + y  
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1b. For your second function, adapt your first function so it can take a single input vector or two inputs as before. For example, add (4,7) and add (c(4, 7, 10)).

```
add <- function(x, y=0) {  
  sum(x + y)  
}
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the ... (dots) argument or the base R function sum()

```
add <- function(x, y, ...) {  
  sum(x, y, ...)  
}
```

```
add(1,2,3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the 'return()' function call.

```
add <- function(x, y=0, z=0) {  
  sum(x, y, z)  
  cat("Is it breaktime yet?" )  
}
```

```
add(10, 100)
```

```
Is it breaktime yet?
```

Q2. Write a generate_dna() function

A useful function here is the “base R” ‘sample()’ function.

```
sample(1:5, size=60, replace="TRUE")
```

```
[1] 2 4 4 1 3 3 4 5 3 4 5 4 5 2 1 3 4 3 2 3 3 2 1 2 4 5 4 5 1 1 5 1 1 4 4 3 1 4  
[39] 5 2 3 4 1 3 5 5 5 1 4 1 5 4 1 3 5 1 1 3 2 5
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, “T” ...

```
sample(x=c("A","C", "T", "G"), size=10, replace=TRUE)
```

```
[1] "G" "G" "G" "A" "T" "A" "A" "G" "A" "A"
```

Q2a. Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return “A”, “T”, “T”, “G”, “A”, “C”.

```
generate_dna <- function(len=10) {
  sample(x=c("A","C", "T", "G"), size=len, replace=TRUE)
}
```

```
generate_dna()
```

```
[1] "G" "T" "C" "G" "G" "G" "G" "C" "A" "T"
```

```
generate_dna(len=10)
```

```
[1] "T" "C" "T" "T" "T" "T" "C" "C" "C" "T"
```

Q2b. Your second version should **optionally** be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”.

```
generate_dna <- function(len=10) {
  sample(x=c("A","C", "T", "G"), size=len, replace=TRUE)
}
```

Functions that could be useful here are ‘paste()’, ‘if()’, ‘cat()’ and ‘return()’.

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(x=c("A","C", "T", "G"), size=len, replace=TRUE)
  #cat("Hello...")
  if(single.element) {
    #cat("is it me you are looking for...")
    ans <- paste(ans, collapse = "")
  }
  return(ans)
}
```

```
generate_dna(single.element=TRUE)
```

```
[1] "TCCGAATGTG"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
>len9
CGAAGGCTG
```

```
cat("hello \n there")
```

```
hello
  there
```

```
generate_dna <- function(len=10, single.element=TRUE) {
  ans <- sample(x=c("A","C", "T", "G"), size=len, replace=TRUE)
  if(single.element) {
    ans <- paste(ans, collapse = "")
  }

  ## Format as FASTA with an ID line
  cat(paste(">len", len, "\n", sep=""))
  cat(ans)
  cat("\n")
  ##

  return(ans)
}
```

```
x <- generate_dna(20)
```

```
>len20
CGAACCCTTTACACAGTGAT
```

Q3. Write a generate_protein() function

****Q3**** “. Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return”WQRTAG”.

```
generate_protein <- function(len=10, single.element=TRUE) {
  ans <- sample(x=c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "V", "W", "Y", "S", "T"), size=len, replace=TRUE)
  if(single.element) {
    ans <- paste(ans, collapse = "")
  }

  ## Format as FASTA with an ID line
```

```

cat(paste(">", len, "\n", sep=""))
cat(ans)
cat("\n")
##
}

```

```

x <- generate_protein(6)

```

```

>6
NRREHI

```

Q4. Generate random protein sequences of length 6 to 13

Adapt and use your 'generate_protein()' function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function 'for()' or 'sapply()' so that you do not have to call generate_protein() eight times by hand.

```

for(l in 6:13) {
  generate_protein(l)
}

```

```

>6
PHVYCY
>7
YCYKAFS
>8
FLWCNUNA
>9
KYDRMNTQY
>10
PLDRNYGQIS
>11
GHINVFPMMNG
>12
GGIDHIYTRCMS
>13
HGLNVCNQTYKRY

```

Q5. BLASTp search against nr — are your peptides “unique in nature”?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100	100	N
7	100	100	N
8	87.50	100	Y
9	88.89	100	Y
10	88.89	90	Y
11	100	73	Y
12	69.23	100	Y
13	72.73	85	Y

Q5a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?

They start to look “unique in nature” when the sequence length is 8 amino acids.

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe

The possible amino acid combinations of shorter peptides (smaller L) is less than that of longer peptides (larger L). Thus, it is more likely for the shorter peptides to match a sequence already known in the protein universe. In other words, very short random peptides are almost always found in nr because they are more likely to match with a sequence in the database.

Q6. Connecting your findings to immunology (MHC class II and T-cell activation)

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Based on the data from Q5, the minimum length for peptide binding is eight amino acids. It might be a bad design choice for the immune system to present very short peptides because it is more likely for them to mistakenly match with a region of a sequence compared to large peptides. The T cells might mistake foreign (non-self) peptides as self peptides. If this happens,

the immune response won't be activated and the potentially dangerous cells won't be killed. It is also possible that the T cells mistake self peptides as non-self. If this happens, the immune response kills their own healthy cells.